Zolatron homebrew micro: the main CPU board

The beating heart of this 6502-based machine, the Zolatron's CPU board is an almost complete single board computer.

Mansfield-Devine

Following

9 min read

.

3 days ago

Listen
Share
More

Press enter or click to view image in full size

The [Zolatron 6502-based homebrew computer] is a backplane design. Each major function of the computer (more or less) is on a separate card, plugged into a backplane board that ties them all together. And so perhaps the best way of describing the computer in detail — at least the hardware — is to delve into the design of each board. And we'll start with the brains of the operation, the CPU board.

At the very least you'd expect the CPU board to contain … well, the CPU. That means the 65C02 in our case, which I chose for my first homebrew project because it's what I grew up with, in the shape of a BBC Micro. The CPU needs a couple of supporting components, include a reset circuit and a clock source, and so those were obvious candidates for this board too. But it would have been wasteful to stop there.

I'd decided on a standard board size of a smidge under 100x100mm, because that's the cheapest size at fab houses. And with just the CPU and a couple of bits and bobs, the board would have been mostly wasted space.

So I asked what else is going into this computer that I'm unlikely to want to change?

The CPU board plugged into the backplane. The three large chips are,

from left to right: RAM, EEPROM and 65C02 CPU.

The whole point of the backplane approach is that if a subsystem of the computer (an interface, for example) doesn't work out, or if I want to improve it or replace it with a different design, I can do that without having to respin the entire computer. That concept has worked out well for me.

As we'll see in future articles about the various boards, at various points I have changed my mind about how to tackle certain aspects of the machine (the serial port is a good example) and have been able to create additional functionality (such as a real-time clock and extended memory) without major disruption or expense.

Boards plugged into the backplane. Note the use of standoffs to separate and stabilise the boards.

The two key features that I knew were not going to change were:

- The ROM
- The RAM

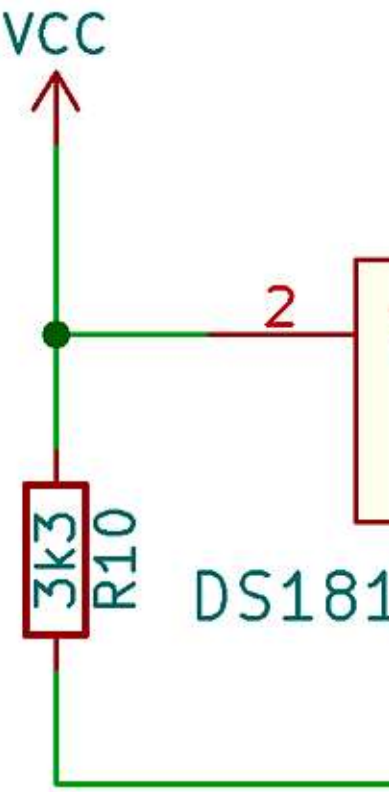
Each of these consists of just a single chip and a bypass capacitor, so on the board they went. But before we get into the major ICs, let's look at some of that supporting hardware.

Resetting

When you first power on, the system needs to be held in reset (ie, with the reset line pulled low) for a brief moment while the CPU gets it socks on.

There are ways of doing this with resistor-capacitor (RC) circuits and an even easier method using a Maxim DS1813 chip. Always a fan of an easy life, that's the direction I went.

Press enter or click to view image in full size



On power up, this holds its `/RST` output pin low for 150ms, which is plenty of time for the CPU to stabilise. So this pins is connected to the system-wide reset line.

Internally, the `/RST` output is open drain, which means the device will pull the signal low when active but otherwise doesn't drive the pin. However, it has its own internal 5.5KΩ pullup resistor and I like to add my own on the reset line. The datasheet says that a 1KΩ external pullup may be needed but I went with a 3.3KΩ resistor. This is in parallel with the internal one giving an effective resistance of around 2KΩ. That should provide a nice, sharp pullup without making the DS1813 work too hard when pulling down.

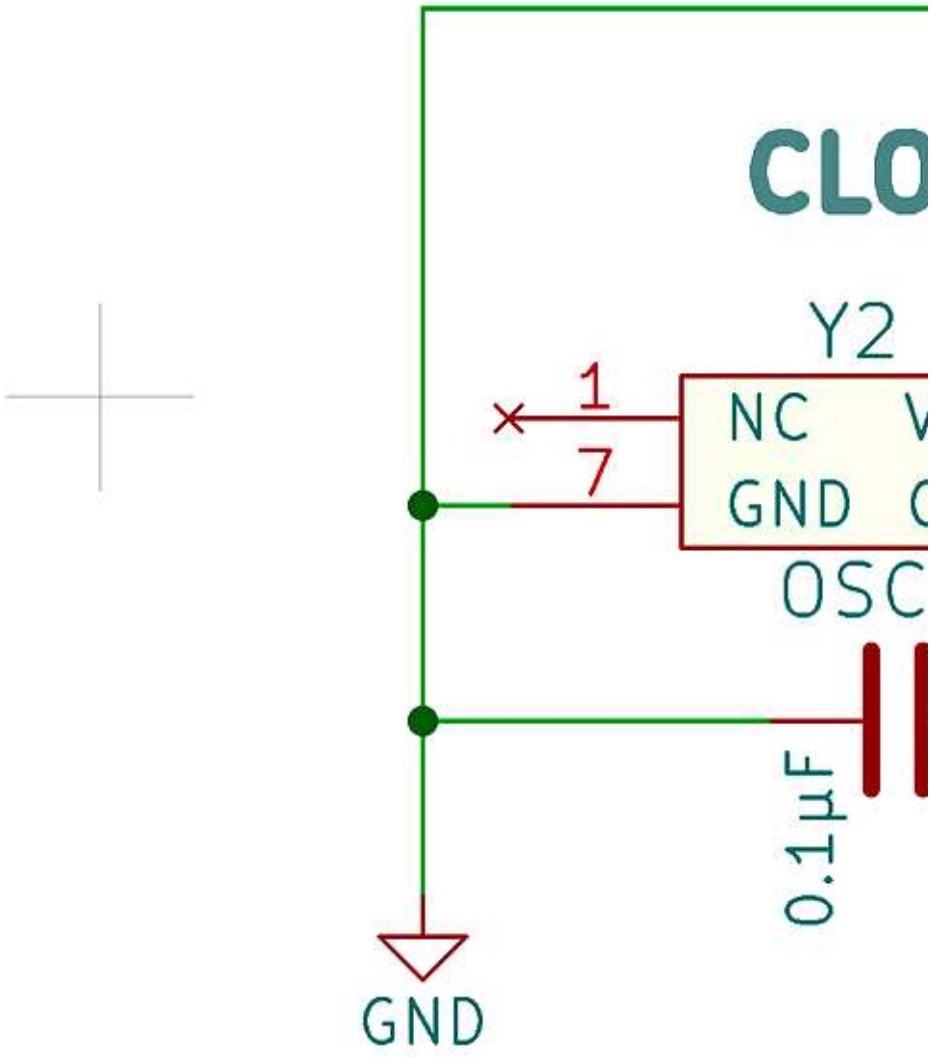
If the `/RST` is pulled down externally — for example as the result of a reset button grounding the line — the DST1813 recognises this and activates, holding `/RST` low for 150ms after the reset button is released. The same is true if the power input drops much below the +5V the chip expects. Depending on the model of the chip you have, if the voltage falls below, say, 4.6V, the chip will assert the `/RST` low for the usual duration.

The DS1813 comes in a package resembling a transistor with three legs (`VCC`, `GND` and `/RST`) and needs no supporting components. It's also available as a surface mount device in SOT-23 form, and I think I'd probably go for that in future.

Clock source

The clock source is the simplest component. It's a plain old oscillator can running at 1MHz, the output of which goes to the CPU's `PHI2` input via a 100Ω resistor. The resistor is there to help prevent unwanted ringing and reflections on the clock line (ie, it provides termination). In my circuit, there's also a capacitor across the power pins for bypassing purposes.

Press enter or click to view image in full size



The oscillator's output also goes to every other component that needs a clock signal. The CPU is not involved in generating the clock, as we'll see later.

If you want to go the full distance and create your own clock source built around a crystal and supporting components,

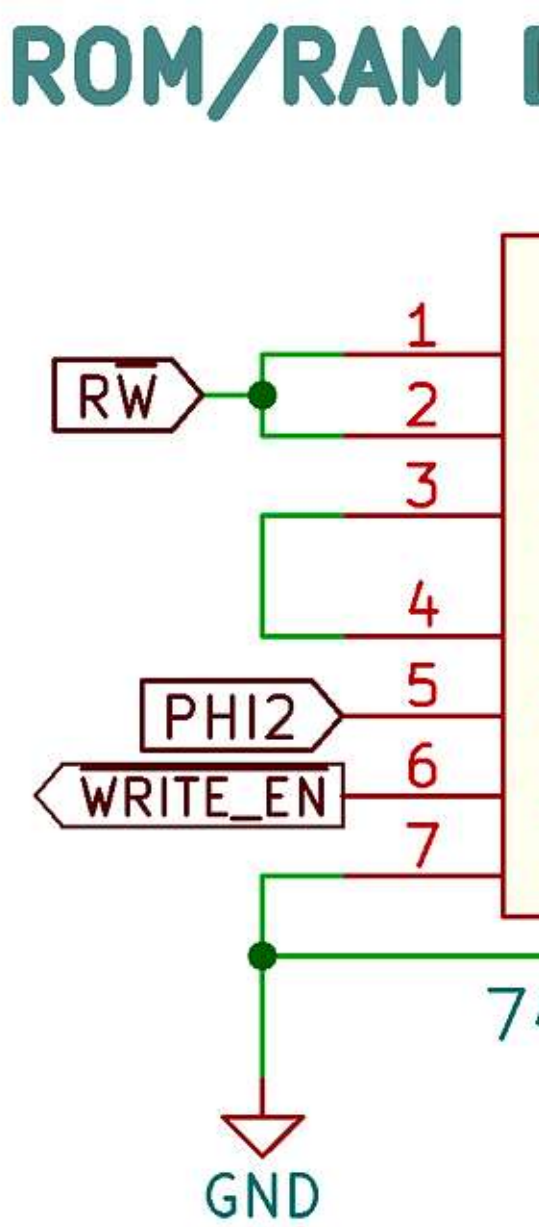
there's a great tutorial here. Ben Eater used a 555-based clock module he'd previously created on a breadboard so that he could run his homebrew design slowly, which certainly has educational benefits.

As a concession to the future me wanting to do silly things with the computer, I did add a couple of header pins so that I can use an external clock source, such as a function generator, if I want to play with different clock speeds. For that reason, I've socketed the oscillator can on the PCB, rather than soldering it in directly.

ROM/RAM decoding

The CPU board includes the address decoding required for the ROM and RAM chips. This is separate to the I/O decoding, but some of the outputs generated are of use beyond the ROM and RAM chips themselves.

Press enter or click to view image in full size



The decoding employs a 74HCT00 IC which boasts four dual-input NAND gates. The three outputs are active-low 'enable' signals — `/ROM_ENABLE`, `/READ_EN` and `/WRITE_EN`. The first is of interest only for this board but the other two have wider applications, and so they are routed out over the backplane. We'll also meet them again when we talk about the RAM and

ROM.

The latter two signals are also both clock-qualified, being active (low) only when the clock signal is high.

CPU connections

Now let's get back to the main event — the CPU. Most of the pins on the 65C02 are spoken for by the data and address buses. But let's take a look at some of the others.

The `PHI2` input (pin 37) gets the clock source signal from the oscillator. The `/IRQ` (interrupt request, pin 4) and `/NMI` (non-maskable interrupt, pin 6) are important signals and, as both are active low, need to be pulled high via resistors. For all the pullups on the CPU I selected 3.3KΩ resistors.

`VCC` (pin 8), `GND` (pin 21) and `/RES` (reset, pin 40) pretty much speak for themselves. I have the `/RES` signal already pulled high in the reset circuit.

Not all of the 65C02's pins are used. In fact, some are now pretty obsolete and have been referred to as '[mystery pins]'. `VPB` (vector pull, pin 1), `MLB` (memory lock, pin 5), `PHI1O` (clock phase 1, pin 3) and `PHI2O` (clock phase 2 output, pin 39) are outputs that you can simply leave unconnected. WDC actually advises against using the clock output signals. Instead, everything should be timed by your clock source.

The `RDY` (ready, pin 2) signal is an input that could be used for single-stepping the processor and for direct memory addressing (DMA). These are not features I need, so I've tied it high with a resistor. `SYNC` (pin 7) is an output also used for single-stepping. In case I want to revisit these sometime, I've brought out both of these

signals to the backplane, but generally you can leave `SYNC` unconnected.

`SOB` (set overflow, pin 38) is of little use and you can tie it high. `BE` (bus enable, pin 36) provides external control of the buses, which is above my pay grade right now (although it *is* something I plan to research as part of my scheme to replace the EEPROM with flash memory). I've also brought this out to the backplane while providing a pullup resistor. Add power and a bypass capacitor and we're done with the CPU.

Don't forget the memory

For the ROM, I chose the AT26C256 EEPROM, which I've discussed in detail in a previous article. The EEPROM lives in a zero insertion force (ZIF) socket because this is a chip that I'm constantly removing and inserting every time I change the ROM code. As hinted above, I have an idea about how to eliminate that tedious process using flash memory, but that's a work in progress.

As we don't write to the EEPROM while it's in the machine, the write enable (`/WE`) pin is just pulled high via a resistor. The output enable (`/OE`) is controlled by the `/READ_EN` signal from the decoding while the chip enable (`/CE`) is, naturally, the only pin in the entire computer to be connected to the `/ROM_ENABLE` output of the decoding.

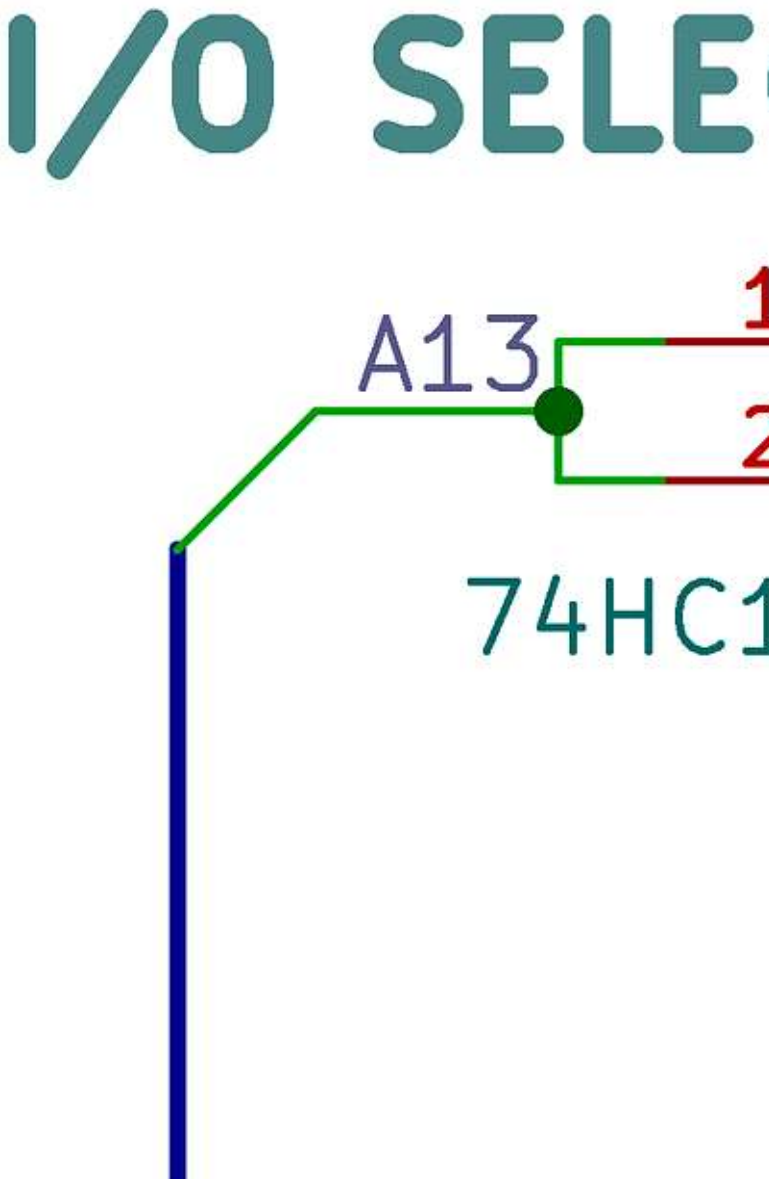
The RAM is the AS6C62256 32KB SRAM chip. The chip enable (`/CE`) is selected by connecting address line `A15` directly to it. The output enable (`/OE`) is controlled by the `/READ_EN` line from the decoding while the write enable (`/WE`) is connected to `/WRITE_EN` (duh).

Both ROM and RAM are also fed the address and data buses, but don't require much besides that.

I/O select

Another simple bit of decoding involves simply invertng the `A13` address signal, which I achieve with a single NAND gate (in the shape of a tiny, surface-mount 74AHCT1G00), tying both the inputs to `A13`.

Press enter or click to view image in full size



The output, which I've labelled `/IO_SEL`, goes out over the backplane. But I'm not using it. This inversion of `A13` is part of my I/O address decoding. But instead of using the signal that's going out over the backplane I've put a NAND gate on each board that requires this

decoding. The I/O address decoding, involving that NAND gate and a 74HCT138 three-to-eight decoder, is something I replicate on every I/O card, which is wasteful.

I have spare lines on the backplane and could have run the decoded signals that way. And I really can't remember why I went the way I did. Something to consider for Version 2 of the Zolatron (which, history informs us, will probably be called the 'Zolatron Plus').

Adding holes

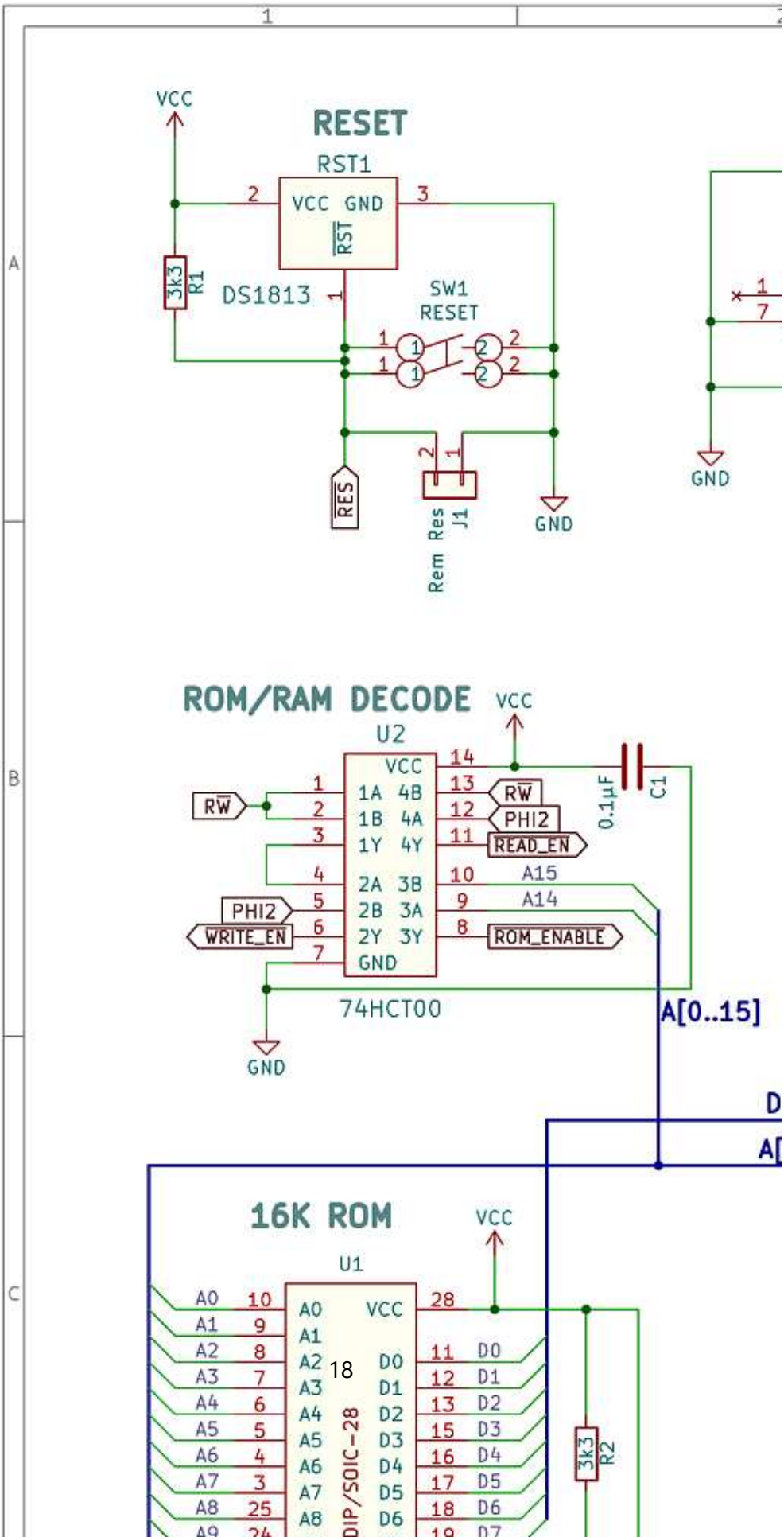
Each board in the Zolatron attaches to the backplane via a 64-pin DIN 41612 connector. This is pretty solid. But to provide extra support I've built the machine into a cage made from MakerBeam aluminium rods. The lower edge of each card rests on a cross-beam.

For a final touch of solidity I added … holes. Each card has four holes in the same places. One board attaches to the next with a standoff (I ended up using mostly just the top, rear hole — the others proved unnecessary). The whole rig is rock-solid and I've never had a problem with wobbly connections.

Outro

With CPU, ROM and RAM on one PCB, this isn't far off being a single-board computer — except that it lacks user input and output. We'll see about that soon.

Press enter or click to view image in full size


RESET
RST1
VCC GND
RST
DS1813
3k3
R1
SW1
RESET
RES
Rem Res
J1
GND
ROM/RAM DECODE
U2
VCC
1A 4B
1B 4A
1Y 4Y
2A 3B
2B 3A
2Y 3Y
GND
RW
PHI2
READ_EN
A15
A14
ROM_ENABLE
WRITE_EN
74HCT00
0.1µF
C1
A[0..15]
16K ROM
U1
A0
A1
A2
A3
A4
A5
A6
A7
A8
D0
D1
D2
D3
D4
D5
D6
D7
DIP/SOIC-28
3k3
R2

The full schematic for the CPU board.

You can find all the stories related to this project on the Zolatron feature page.

There is also a GitHub Zolatron repo with the code, datasheets and other documents.

Steve Mansfield-Devine is a freelance writer, tech journalist and photographer. You can find photography portfolio at Zolachrome, buy his books and e-books, or follow him on Bluesky or Mastodon.

You can also buy Steve a coffee. He'd like that.

Technology

Electronics